



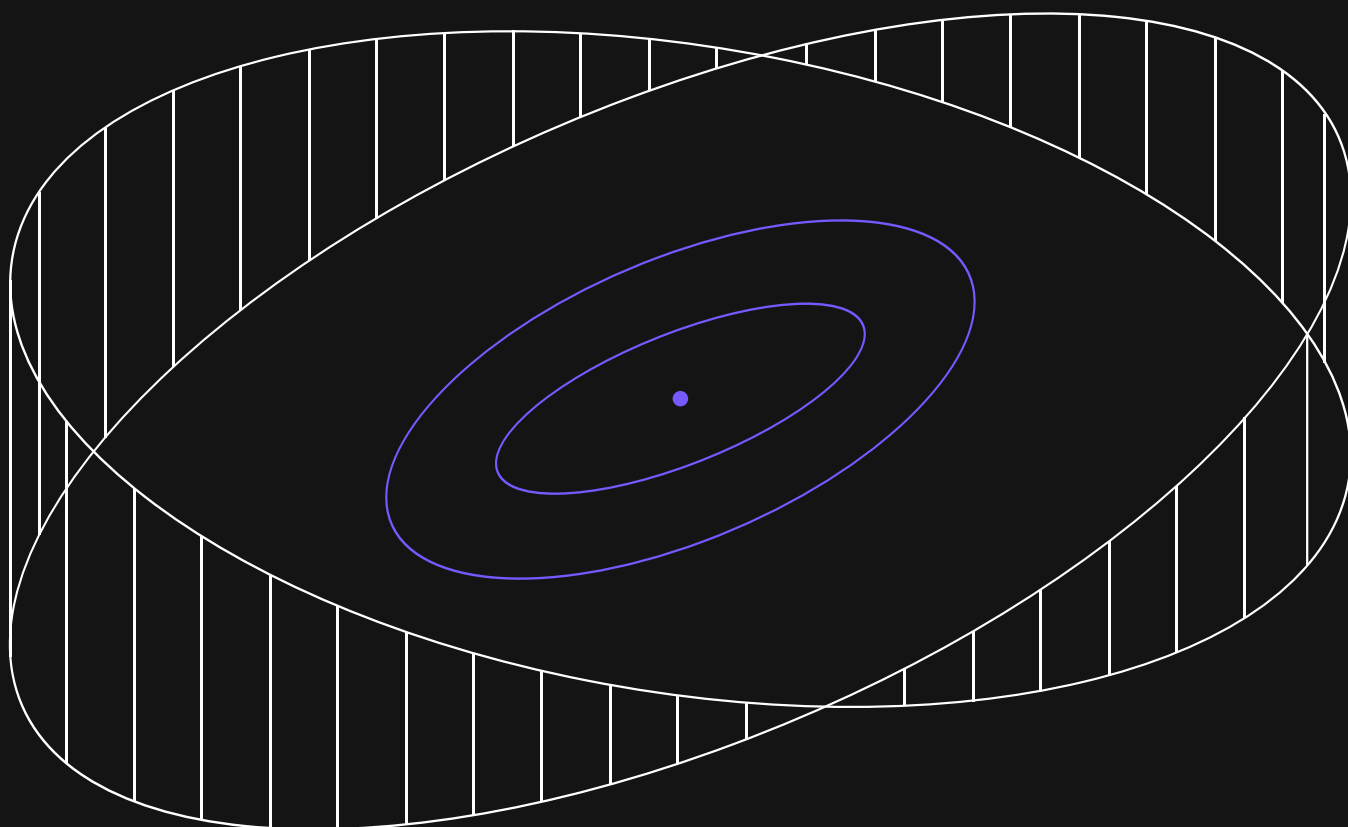
Машинное обучение

Лонгрид

Глубокое обучение

02

Оптимизация в глубоком обучении



Содержание

1. Оптимизация большой конечной суммы	3
1.1. Стоимость прямого и обратного проходов	4
2. Стохастический градиентный спуск	5
2.1. Несмещённость	5
2.2. Стоимость итерации	5
3. Mini-batch SGD	5
3.1. Размер batch	6
3.2. Epoch	6
4. Что происходит около минимума	6
4.1. Постоянный learning rate	7
5. Learning rate и сходимость SGD	7
5.1. Почему line search неудобен	7
5.2. Оценка для постоянного шага	8
5.3. Убывающий learning rate	8
6. Невыпуклость и выбор минимума	9
6.1. Узкие и широкие минимумы	9
7. Расписания learning rate	9
7.1. Linear decay	9
7.2. Exponential decay	10
7.3. Cyclical learning rate	10
7.4. Cosine annealing	10
7.5. Warmup	10
8. Обусловленность и методы второго порядка	12
8.1. Метод Ньютона	12
8.2. Квазиньютоновские методы	12
9. Momentum	13
10. AdaGrad, RMSProp и Adam	13
10.1. AdaGrad	13
10.2. RMSProp	14
10.3. Adam	14
11. L_2-регуляризация, weight decay и AdamW	14
11.1. L_2 -регуляризация	14
11.2. Почему в Adam это не то же самое	15
11.3. AdamW	15
12. Современные варианты оптимизации	15
12.1. Constant learning rate with cooldown	15
12.2. Schedule-Free Optimizer	16
12.3. SignSGD, Signum и Lion	16
12.4. Shampoo и SOAP	16
12.5. Muon	17
12.6. Сравнение современных методов	18
13. Как выбирать оптимизатор	19
14. Главное	20

Лекция 4. Оптимизация в глубоком обучении

О чём эта лекция

Обучение нейронной сети сводится к минимизации средней потери по большой выборке. В этой лекции разбираются полный градиент, SGD и mini-batch SGD, выбор learning rate, поведение метода около минимума, обусловленность задачи и основные семейства оптимизаторов: Momentum, AdaGrad, RMSProp, Adam и AdamW. В конце собраны современные варианты расписаний и оптимизаторов.



Граница конспекта

Здесь рассматривается оптимизация параметров. Dropout, Batch Normalization, инициализация, репараметризация и остаточные связи разобраны в лекции 3 о стабильности и регуляризации и здесь не повторяются.

1. Оптимизация большой конечной суммы



Обозначения

- x_i — объект обучающей выборки, N — число объектов в обучающей выборке;
- $w \in \mathbb{R}^d$ — все обучаемые параметры модели, d — их число;
- $L_i(w)$ — потеря модели на объекте x_i , $F(w)$ — средняя потеря, или эмпирический риск;
- w_k — параметры на итерации k , $w^\star$ — точка минимума;
- α_k — learning rate на итерации k ;
- g_k — стохастическая оценка градиента, I_k — mini-batch на итерации k ;
- $\odot$ — покомпонентное умножение.

Learning rate здесь обозначается α_k ; в лекциях 2 и 3 для него использовалось обозначение η , оно же встречается в разделе 12.2 в записи Schedule-Free. Буква d в этой лекции означает число параметров, а не размерность входа.

Пусть потеря на объекте x_i равна $L_i(w)$. Обучение модели записывается как минимизация средней потери:

$$F(w) = \frac{1}{N} \sum_{i=1}^N L_i(w) \longrightarrow \min_w, \quad \nabla F(w) = \frac{1}{N} \sum_{i=1}^N \nabla L_i(w).$$

Если выборка содержит сотни тысяч или миллионы объектов, вычислять эту сумму на каждом шаге слишком дорого.

1.1. СТОИМОСТЬ ПРЯМОГО И ОБРАТНОГО ПРОХОДОВ

Обозначим через Q стоимость прямого прохода по одному объекту, а через d — число параметров.

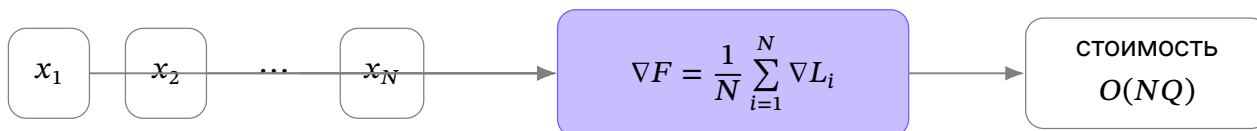
Вычисление	Порядок стоимости
Потеря на одном объекте $L_i(w)$	$O(Q)$
$\nabla L_i(w)$ конечными разностями	$O(Qd)$
$\nabla L_i(w)$ обратным распространением	$O(Q)$
Полная функция $F(w)$	$O(NQ)$
Полный градиент $\nabla F(w)$	$O(NQ)$

Таблица 1. Порядок стоимости базовых вычислений

Reverse-mode automatic differentiation вычисляет градиент по всем d параметрам за время того же порядка, что и прямой проход, а не требует отдельного запуска для каждой координаты.

Для линейного слоя прямой проход содержит умножение на W , а обратный — умножение на W^T . Поэтому полный обратный обход графа сопоставим по стоимости с прямым.

Полный градиент



Mini-batch

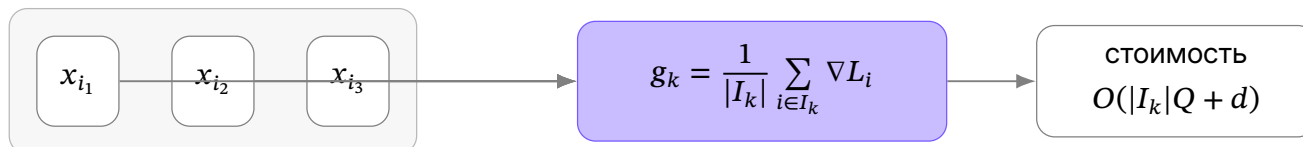


Рис. 1. Полный градиент суммирует вклады всех N объектов, поэтому стоимость итерации растёт с размером выборки; mini-batch усредняет вклады небольшого подмножества, и стоимость шага от N не зависит



Требование к методу

Стоимость одной итерации должна зависеть от обработки небольшого числа объектов, но не от полного размера выборки N .

2. Стохастический градиентный спуск

На итерации k равномерно выбирается один объект:

$$i_k \sim \text{Unif}\{1, \dots, N\}, \quad g_k = \nabla L_{i_k}(w_k), \quad w_{k+1} = w_k - \alpha_k g_k.$$

2.1. НЕСМЕЩЁННОСТЬ

При равномерном выборе объекта

$$\begin{aligned} \mathbb{E}[g_k \mid w_k] &= \mathbb{E}[\nabla L_{i_k}(w_k) \mid w_k] \\ &= \frac{1}{N} \sum_{i=1}^N \nabla L_i(w_k) \\ &= \nabla F(w_k). \end{aligned}$$

Следовательно, g_k — случайная, но несмещённая оценка полного градиента.

2.2. СТОИМОСТЬ ИТЕРАЦИИ

Действие	Стоимость
Выбрать объект	$O(1)$
Вычислить $\nabla L_{i_k}(w_k)$	$O(Q)$
Обновить d параметров	$O(d)$

Таблица 2. Из чего складывается стоимость одного шага SGD

Один шаг SGD намного дешевле полного градиентного шага. Цена этой экономии — шум в направлении движения и большее число итераций.

3. Mini-batch SGD

На практике градиент оценивают не по одному объекту, а по набору I_k :

$$g_k = \frac{1}{|I_k|} \sum_{i \in I_k} \nabla L_i(w_k), \quad w_{k+1} = w_k - \alpha_k g_k.$$

Если объекты выбираются равномерно, mini-batch-градиент также остаётся несмещённой оценкой $\nabla F(w_k)$.

3.1. РАЗМЕР BATCH

При независимом семплировании стандартное отклонение средней оценки имеет порядок:

$$\text{std}(g_k) \propto \frac{1}{\sqrt{|I_k|}}.$$

Увеличение batch size:

- уменьшает шум градиента;
- лучше использует параллельные вычисления;
- увеличивает стоимость одной итерации;
- даёт убывающую отдачу: увеличение batch в несколько раз не уменьшает шум во столько же раз.

Стоимость шага составляет $O(|I_k|Q + d)$. На практике размеры batch часто измеряются десятками или сотнями объектов: переход к тысячам объектов не даёт пропорционального уменьшения шума.

3.2. ЕРОСН

Один проход по всей обучающей выборке называется эпохой. Типичный цикл обучения показан на схеме:

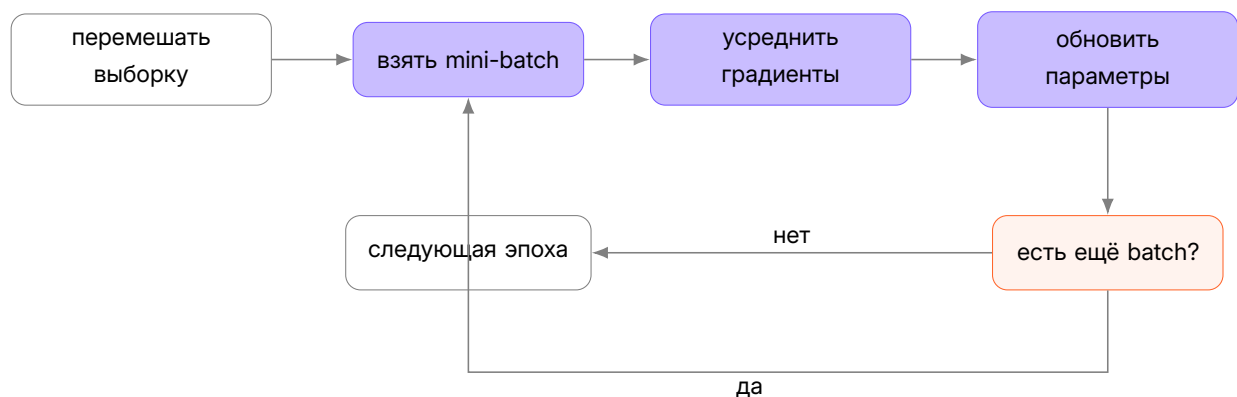


Рис. 2. Одна эпоха: выборка перемешивается, затем по очереди берутся mini-batch, и на каждом выполняется один шаг оптимизации

4. Что происходит около минимума

Рассмотрим одномерную линейную регрессию:

$$F(w) = \frac{1}{N} \sum_{i=1}^N (y_i - wx_i)^2.$$

Минимум отдельной функции $L_i(w) = (y_i - wx_i)^2$ обычно не совпадает с минимумом средней функции $F(w)$. Поэтому в точке $w^\star$

$$\nabla F(w^\star) = 0, \quad \text{но обычно} \quad \nabla L_i(w^\star) \neq 0.$$

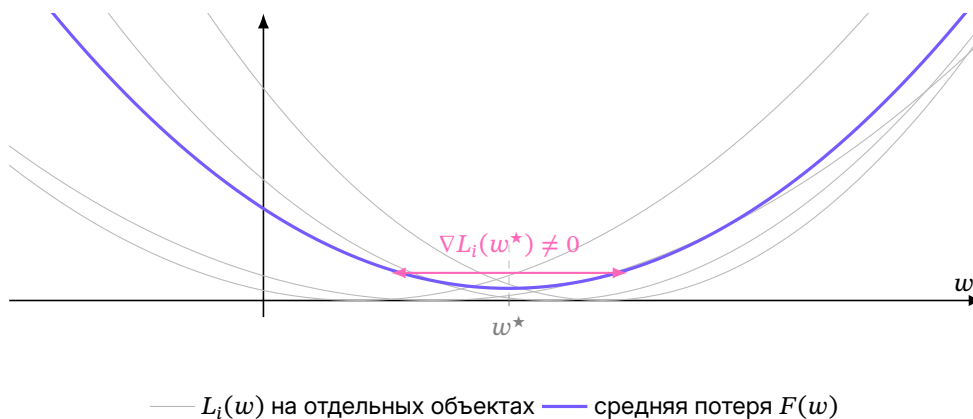


Рис. 3. В минимуме средней функции стохастические градиенты отдельных объектов направлены в разные стороны и не обращаются в ноль --- поэтому шаг по одному объекту уведет метод из точки w^*

4.1. ПОСТОЯННЫЙ LEARNING RATE

Из-за шума SGD с постоянным learning rate не обязан остановиться в точке минимума. Метод продолжает колебаться в её окрестности:

- большой α быстрее приводит в область решения, но оставляет широкие колебания;
- маленький α уменьшает колебания, но замедляет движение.

5. Learning rate и сходимость SGD

5.1. ПОЧЕМУ LINE SEARCH НЕУДОБЕН

В детерминированной оптимизации шаг можно подбирать по значению полной функции. В SGD точное значение $F(w)$ требует прохода по всей выборке, поэтому line search разрушает преимущество дешёвой итерации.

По той же причине условие

$$\|\nabla F(w_k)\| \leq \varepsilon$$

неудобно как практический критерий остановки: вычисление полного градиента дорого. Обычно обучение останавливают:

- после заданного числа эпох;
- по ограничению времени;
- по качеству на validation;
- при отсутствии улучшения validation-метрики.

Learning rate можно уменьшать, когда validation-метрика перестаёт улучшаться.

5.2. ОЦЕНКА ДЛЯ ПОСТОЯННОГО ШАГА

Для выпуклой функции одна из стандартных оценок имеет вид:

$$\mathbb{E}[F(\bar{w}_k)] - F(w^\star) \leq \frac{\|w_0 - w^\star\|^2 + \sum_{i=0}^k \alpha_i^2 \mathbb{E}\|g_i\|^2}{2 \sum_{i=0}^k \alpha_i}, \quad \bar{w}_k = \frac{\sum_{i=0}^k \alpha_i w_i}{\sum_{i=0}^k \alpha_i},$$

где $\bar{w}_k$ — усреднённая по шагам точка. Если $\alpha_i = h$, $\|w_0 - w^\star\| \leq R$ и $\mathbb{E}\|g_i\|^2 \leq G^2$, то верна оценка:

$$\mathbb{E}[F(\bar{w}_k)] - F(w^\star) \leq \frac{R^2}{2h(k+1)} + \frac{hG^2}{2}.$$

Первое слагаемое уменьшается с числом итераций, а второе задаёт ненулевую шумовую окрестность. Уменьшение h сужает эту окрестность, но замедляет исчезновение первого слагаемого.

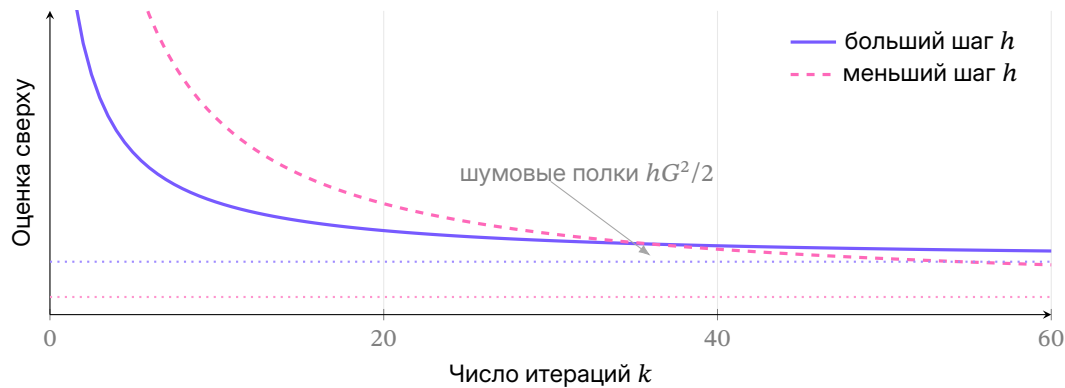


Рис. 4. Оценка для постоянного шага складывается из убывающего слагаемого $R^2/(2h(k+1))$ и постоянной шумовой полки $hG^2/2$: больший шаг быстрее сбивает первое слагаемое, но оставляет более широкую окрестность



SGD не обязан быть методом спуска на каждой итерации

Отдельный стохастический шаг может увеличить полную функцию F . Оценки формулируются в среднем, а не как монотонное убывание на каждом шаге.

5.3. УБЫВАЮЩИЙ LEARNING RATE

Классические условия сходимости имеют вид

$$\sum_{k=0}^{\infty} \alpha_k = \infty, \quad \sum_{k=0}^{\infty} \alpha_k^2 < \infty.$$

Первое условие не даёт методу остановиться слишком рано, второе постепенно подавляет шум. Более слабое условие записывается в виде:

$$\frac{\sum_{i=0}^k \alpha_i^2}{\sum_{i=0}^k \alpha_i} \rightarrow 0.$$

Для степенного расписания $\alpha_k = C/(k+1)^\tau$ классические условия выполняются при $\frac{1}{2} < \tau \leq 1$. При $\tau = 1$ получается скорость порядка $O(1/\log k)$, а граничный выбор $\tau = 1/2$ даёт более слабую оценку:

$$O\left(\frac{\log k}{\sqrt{k}}\right) = \tilde{O}\left(\frac{1}{\sqrt{k}}\right).$$

Чтобы получить точность ε , требуется примерно $k \sim 1/\varepsilon^2$ итераций. Поэтому высокая точность стохастического метода может быть практически недостижима.

Шум и ранняя остановка при этом могут играть роль неявной регуляризации: модель не обязана доходить до точного минимума обучающей функции.

6. Невыпуклость и выбор минимума

Функция потерь нейронной сети невыпукла. У неё могут быть локальные минимумы, седловые точки, плоские направления и области с разной кривизной. Гарантии нахождения глобального минимума обычно отсутствуют, поэтому результат оценивают не только по обучающей функции, но и по validation.

6.1. УЗКИЕ И ШИРОКИЕ МИНИМУМЫ

В стохастической постановке отдельные объекты могут задавать немного разные локальные цели. Широкий минимум интуитивно соответствует области параметров, где небольшие изменения w мало меняют значение функции. Такая область может содержать несколько близких локальных минимумов.

Отсюда практическая идея:

- 1) начать с достаточно большого learning rate, чтобы не зафиксироваться в узкой области;
- 2) попасть в широкую область низких значений функции;
- 3) уменьшить learning rate и уточнить решение.

Изменение learning rate во время обучения называется scheduling.

7. Расписания learning rate

Пусть t — номер итерации, T — запланированное число итераций.

7.1. LINEAR DECAY

$$\alpha_t = \alpha_0 \left(1 - \frac{t}{T}\right).$$

Learning rate линейно уменьшается от α_0 к нулю.

7.2. EXPONENTIAL DECAY

$$\alpha_t = \alpha_0 e^{-\lambda t}.$$

Темп уменьшения задаётся коэффициентом λ ; с коэффициентом L_2 -регуляризации из раздела 11 он не связан.

7.3. CYCLICAL LEARNING RATE

$$\alpha_t = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \text{scale}(t).$$

Здесь $\text{scale}(t)$ — периодическая функция со значениями от 0 до 1, например треугольная: learning rate циклически проходит путь от нижней границы к верхней и обратно.

7.4. COSINE ANNEALING

$$\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min}) \left(1 + \cos \frac{T_{\text{cur}}}{T_i} \pi \right).$$

На практике cosine annealing часто используют без перезапусков и обновляют на каждой итерации. Тогда T_i — полное число итераций, а T_{cur} — текущая итерация.

7.5. WARMUP

При линейном warmup

$$\alpha_t = \alpha_0 \min \left(1, \frac{t}{T_{\text{warmup}}} \right).$$

В начале learning rate плавно увеличивается, после чего применяется основное расписание. Частая комбинация — CosineAnnealing + Warmup.

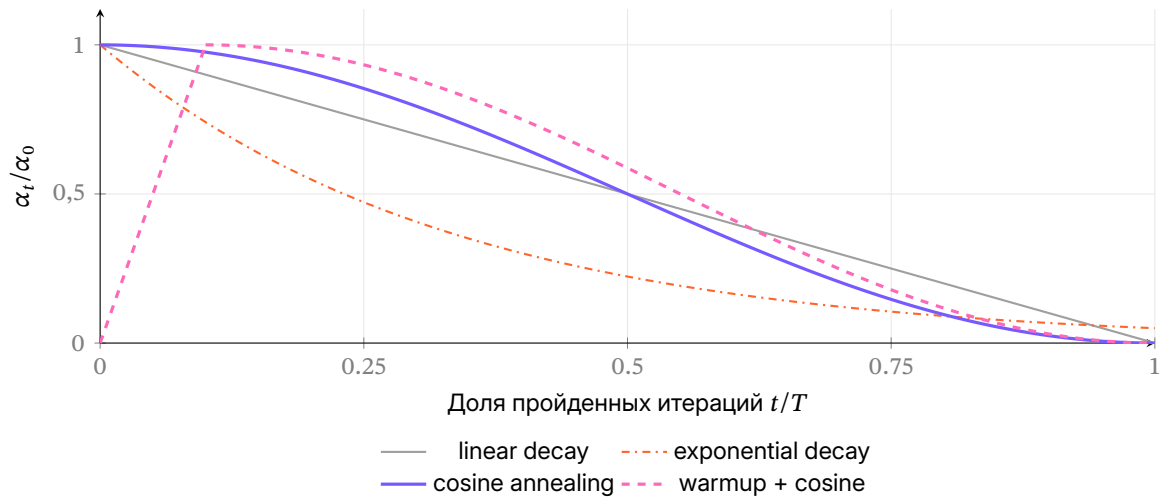


Рис. 5. Основные расписания в одинаковом масштабе: линейное и экспоненциальное убывание, косинусный отжиг и та же косинусная кривая с линейным разогревом в начале

Warmup используют по трём причинам.

1. Фактический размер шага определяется произведением learning rate на величину градиента, а в начале градиенты могут быть большими.
2. У адаптивных методов первые оценки моментов ещё шумны, и небольшой шаг позволяет им стабилизироваться.
3. Допустимый шаг связан с кривизной: случайная инициализация может попасть в область с большой максимальной собственной величиной гессиана.

Для квадратичной задачи устойчивый шаг удовлетворяет условию:

$$0 < \alpha < \frac{2}{\lambda_{\max}}, \quad \text{а консервативный выбор имеет порядок} \quad \alpha \leq \frac{1}{\lambda_{\max}}.$$

8. Обусловленность и методы второго порядка

Если линии уровня сильно вытянуты, одинаковый шаг по всем направлениям приводит к зигзагообразной траектории: вдоль направления большой кривизны шаг слишком велик, а вдоль пологого — слишком мал.

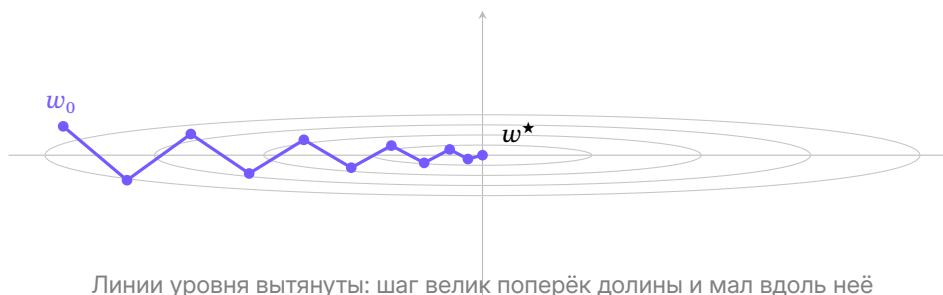


Рис. 6. Плохая обусловленность: градиентный метод с одинаковым шагом по всем направлениям идёт зигзагом поперёк вытянутой долины вместо движения вдоль неё

8.1. МЕТОД НЬЮТОНА

Метод Ньютона использует гессиан:

$$w_{k+1} = w_k - \alpha_k [\nabla^2 F(w_k)]^{-1} \nabla F(w_k).$$

Он корректирует направление с учётом локальной кривизны. Но для нейронной сети гессиан имеет размер $d \times d$, поэтому его хранение, построение и обращение слишком дороги.

8.2. КВАЗИНЬЮТОНОВСКИЕ МЕТОДЫ

Квазиньютоновские методы приближают информацию о кривизне, не строя полный гессиан. L-BFGS эффективен в детерминированных задачах, однако в mini-batch-режиме возникают две проблемы:

- кривизна, оценённая по шумному batch, ненадёжна;
- сложная итерация может стоить дороже нескольких обычных шагов первого порядка.

9. Momentum

Momentum добавляет к текущему градиентному шагу движение с предыдущей итерации:

$$w_{k+1} = w_k - \alpha_k g_k + \beta_k (w_k - w_{k-1}).$$

Ту же идею записывают через накопленное направление:

$$\begin{aligned} v_k &= \beta v_{k-1} + (1 - \beta) g_k, \\ w_{k+1} &= w_k - \alpha_k v_k. \end{aligned}$$



Две записи совпадают с точностью до масштаба шага

Во второй форме множитель $(1 - \beta)$ уже входит в v_k , поэтому при одном и том же α_k шаги различаются в $(1 - \beta)$ раз. Переход от одной записи к другой меняет смысл константы α_k , а не поведение метода.

Разворачивая рекурсию, получаем экспоненциально взвешенное среднее прошлых градиентов:

$$v_k = (1 - \beta) \sum_{j=0}^k \beta^{k-j} g_j.$$

Momentum сглаживает шум, подавляет колебания в направлениях большой кривизны и ускоряет движение вдоль устойчивого направления.

10. AdaGrad, RMSProp и Adam

Эти методы выбирают разные масштабы шага для разных координат.

10.1. ADAGRAD

AdaGrad накапливает квадраты всех прошлых градиентов:

$$s_k = s_{k-1} + g_k \odot g_k, \quad w_{k+1} = w_k - \alpha \frac{g_k}{\sqrt{s_k} + \varepsilon}.$$

Координаты с большими накопленными градиентами получают меньший шаг. Но s_k только растёт, поэтому learning rate может со временем стать слишком малым; для невыпуклых задач это неудобно.

10.2. RMSPROP

RMSProp заменяет сумму экспоненциальным средним:

$$s_k = \beta_2 s_{k-1} + (1 - \beta_2) g_k \odot g_k, \quad w_{k+1} = w_k - \alpha \frac{g_k}{\sqrt{s_k} + \varepsilon}.$$

Старые градиенты постепенно забываются, поэтому знаменатель не обязан неограниченно расти.

10.3. ADAM

Adam объединяет momentum и RMSProp:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k, \\ v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k \odot g_k.$$

Если $m_0 = v_0 = 0$, первые оценки смещены к нулю. Поэтому используют поправку:

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}, \quad w_{k+1} = w_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}.$$



Одна буква в двух ролях

В разделе о Momentum через v_k обозначено накопленное направление, а здесь — второй момент градиента, то есть та же величина, которую AdaGrad и RMSProp называют s_k . Это стандартные обозначения соответствующих методов, но в пределах лекции их стоит различать по контексту.



Родственные варианты

AMSGrad изменяет правило накопления второго момента, а AdaMax использует другую норму. Это разные модификации Adam.

11. L_2 -регуляризация, weight decay и AdamW

11.1. L_2 -РЕГУЛЯРИЗАЦИЯ

К функции добавляют штраф за большие параметры:

$$F_\lambda(w) = F(w) + \frac{\lambda}{2} \|w\|^2.$$

Это связано с постановкой, где норма параметров ограничена. При этом разные слои могут иметь разные естественные масштабы, поэтому один общий коэффициент λ не всегда одинаково действует на все параметры.

Для обычного SGD шаг по F_λ равен:

$$\begin{aligned} w_{k+1} &= w_k - \alpha_k(\nabla F(w_k) + \lambda w_k) \\ &= (1 - \lambda \alpha_k)w_k - \alpha_k \nabla F(w_k). \end{aligned}$$

Такое мультипликативное уменьшение параметров называют weight decay.

11.2. ПОЧЕМУ В ADAM ЭТО НЕ ТО ЖЕ САМОЕ

В обычном SGD L_2 -регуляризация и weight decay приводят к одному обновлению. В адаптивном методе штраф входит в тот же градиент, по которому строятся оба момента. Пусть

$$\tilde{g}_k = \nabla F(w_k) + \lambda w_k,$$

и моменты $\hat{m}_k, \hat{v}_k$ считаются по $\tilde{g}_k$. Тогда обновление Adam имеет вид

$$w_{k+1} = w_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon},$$

то есть вклад λw_k делится на $\sqrt{\hat{v}_k} + \varepsilon$ вместе с градиентом и вдобавок сам влияет на второй момент. Координаты с большим вторым моментом получают меньшее затухание, и эквивалентность теряется.

11.3. ADAMW

AdamW отделяет адаптивный шаг Δ_k от уменьшения параметров:

$$w_{k+1} = w_k - \alpha_k(\Delta_k + \lambda w_k) = (1 - \alpha_k \lambda)w_k - \alpha_k \Delta_k.$$

Weight decay применяется непосредственно к параметрам и не зависит от покомпонентного масштаба адаптивного градиента.

12. Современные варианты оптимизации

12.1. CONSTANT LEARNING RATE WITH COOLDOWN

Cosine annealing требует заранее знать полное число итераций. Кроме того, качество модели в середине обучения при большом learning rate ещё не соответствует качеству после финального уменьшения шага.

Отсюда простая схема: warmup, затем постоянный learning rate, затем cooldown — уменьшение шага перед завершением или оценкой. По разобранным результатам такая схема работает не хуже привычного cosine annealing с warmup и удобнее, когда длительность обучения заранее не известна.

12.2. SCHEDULE-FREE OPTIMIZER

В теории стохастической оптимизации усреднение траектории часто даёт более устойчивую точку, тогда как расписания улучшают последнюю итерацию, но требуют знать горизонт обучения.

SGD + Momentum-версия Schedule-Free использует три последовательности параметров:

$$y_t = (1 - \beta)z_t + \beta x_t, \quad z_{t+1} = z_t - \eta \nabla L_{i_t}(y_t), \quad x_{t+1} = (1 - c_{t+1})x_t + c_{t+1}z_{t+1}.$$

Усреднение встраивается в сам оптимизатор, поэтому зависимость от внешнего расписания уменьшается. При этом расписание всё ещё может улучшить качество обучения.

Практическая деталь: последовательности, по которым считаются градиенты и по которым оценивается качество, здесь разные. Поэтому при валидации и сохранении чекпоинта нужно переключать режим и брать усреднённые параметры, а не те, в которых считался градиент.

12.3. SIGNSGD, SIGNUM И LION

При распределённом обучении с очень большим batch градиенты считаются на нескольких устройствах, и их передача может стать узким местом.

Можно передавать только знак координат градиента и выбирать общий знак голосованием большинства. Это уменьшает объём передаваемых данных в 32 раза или в 16 раз — в зависимости от исходной точности.

- SignSGD использует знак градиента.
- Signum добавляет инерцию.
- Lion добавляет адаптивность.

Использование только знака градиента может работать не только как средство распределённой коммуникации, но и как способ обучения; в некоторых задачах оно также даёт регуляризующий эффект.

12.4. SHAMPOO И SOAP

Метод Ньютона быстро сходится благодаря информации о гессиане, но вычислительно дорог. Отсюда возникает идея аппроксимировать кривизну по информации о градиентах.

Shampoo — метод с приближением информации второго порядка. Он показывает хорошее качество, но требует регулярных дорогих разложений. Если пересчитывать медленные статистики слишком редко, они начинают расходиться с величинами, обновляемыми на каждой итерации.

SOAP добавляет связь с прошлым для регулярно обновляемых параметров по аналогии с Adam и, по разобранным результатам, показывает хорошее качество без увеличения времени обучения на порядки.

Идея SOAP: повернуть градиент в базис кривизны, применить там обычный Adam и вернуть полученное обновление обратно. Для матрицы весов накапливают две статистики — по строкам и по столбцам:

$$L_t = \beta_2 L_{t-1} + (1 - \beta_2) G_t G_t^\top, \quad R_t = \beta_2 R_{t-1} + (1 - \beta_2) G_t^\top G_t.$$

Периодически — не на каждом шаге, потому что это дорого — для них строят собственные базисы

$$L_t = Q_L \Lambda_L Q_L^T, \quad R_t = Q_R \Lambda_R Q_R^T,$$

и переносят градиент в новые координаты: $G'_t = Q_L^T G_t Q_R$. В повернутом базисе работает обычный Adam:

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) G_t, \quad M'_t = Q_L^T M_t Q_R,$$

$$V_t = \beta_2 V_{t-1} + (1 - \beta_2) (G'_t \odot G'_t), \quad N'_t = \frac{M'_t}{\sqrt{V_t} + \varepsilon}.$$

Остаётся вернуть обновление в исходные координаты и сделать шаг:

$$N_t = Q_L N'_t Q_R^T, \quad W_{t+1} = W_t - \eta_t N_t.$$

По сравнению с Adam схема богаче: она учитывает связи между строками и столбцами матрицы, а не только отдельные координаты. Цена — дополнительная память под L_t , R_t , Q_L , Q_R и периодические разложения матриц.

12.5. MUON

Adam и его варианты работают покоординатно: каждый вес масштабируется независимо от остальных. Но веса скрытых слоёв естественно организованы в матрицы, и покоординатная обработка эту структуру игнорирует. Shampoo и SOAP учитывают её через приближение кривизны, но платят регулярными разложениями матриц.

Muon (MomentUm Orthogonalized by Newton–Schulz) заходит с другой стороны: он берёт обычный шаг SGD с моментом и заменяет матрицу обновления на ближайшую к ней ортогональную.

$$M_t = \mu M_{t-1} + \nabla L_t(W_{t-1}), \quad O_t = \text{Ortho}(M_t), \quad W_t = W_{t-1} - \eta_t O_t.$$

Ортогонализация здесь понимается как задача о ближайшей ортогональной матрице:

$$\text{Ortho}(M) = \arg \min_O \{\|O - M\|_F : O^T O = I \text{ или } O O^T = I\}.$$

Если $M = U \Sigma V^T$ — сингулярное разложение, то решением будет UV^T : сингулярные направления сохраняются, а все сингулярные числа становятся одинаковыми.

Смысл в следующем. В накопленном моменте обычно доминируют несколько направлений с большими сингулярными числами, и шаг фактически идёт только по ним. Ортогонализация выравнивает вклад направлений: редкие направления получают тот же масштаб, что и доминирующие. Это ещё один способ бороться с плохой обусловленностью — не покоординатный, как в Adam, а спектральный.

Считать SVD на каждом шаге слишком дорого. Вместо этого используют итерацию Ньютона–Шульца: берут $X_0 = M / \|M\|_F$ и несколько раз применяют

$$X_k = a X_{k-1} + b (X_{k-1} X_{k-1}^T) X_{k-1} + c (X_{k-1} X_{k-1}^T)^2 X_{k-1}.$$

Коэффициенты a , b , c подобраны так, чтобы сингулярные числа подтягивались к единице за небольшое число шагов; на практике берут около пяти. Итерация состоит только из матричных умножений, не требует SVD и устойчиво считается в bfloat16. Точной ортогональности при этом не достигается —

сингулярные числа остаются в окрестности единицы, — но качества оптимизатора это не портит.

Muon применяют только к двумерным весам скрытых слоёв. Эмбединги, выходной слой, смещения и параметры нормализаций обучают обычным методом, как правило AdamW.

На практике к базовой схеме добавляют несколько уточнений. Момент берут в форме Нестерова: ортогонализируют не сам M_t , а комбинацию $\tilde{M}_t = \nabla L_t(W_{t-1}) + \mu M_t$. Weight decay вносят прямо в шаг, а само обновление домножают на масштаб $s(m, n)$, зависящий от формы матрицы:

$$W_{t+1} = (1 - \eta_t \lambda) W_t - \eta_t s(m, n) O_t.$$

Без weight decay нормы весов со временем растут и выходят за комфортный диапазон bfloat16. Масштаб $s(m, n)$ нужен потому, что у ортогональной матрицы норма фиксирована и не зависит от размера слоя. Один из используемых вариантов подбирает s так, чтобы величина обновления совпадала с типичной для AdamW — тогда уже настроенный learning rate переносится без повторного подбора.

В таком виде Muon применяли при обучении больших языковых моделей: например, модель Kimi K2 обучена вариантом MuonClip, где к Muon добавлены weight decay, согласование масштаба обновления и ограничение роста attention-логитов.



Про оценки выигрыша

В отчётах по масштабированию Muon сообщается о заметном выигрыше по вычислительной эффективности относительно AdamW. Это активно развивающаяся область, и величина выигрыша зависит от архитектуры, режима обучения и аккуратности настройки, поэтому Muon, как и остальные варианты, проверяют по validation на своей задаче.

12.6. СРАВНЕНИЕ СОВРЕМЕННЫХ МЕТОДОВ

Все разобранные методы удобно записать единообразно. Оптимизатор — это преобразование $\mathcal{T}_t$, которое по истории градиентов строит направление шага:

$$\Delta_t = \mathcal{T}_t(G_1, \dots, G_t), \quad W_{t+1} = (1 - \eta_t \lambda) W_t - \eta_t \Delta_t.$$

Один и тот же градиент методы превращают в разные направления; различаются они именно видом $\mathcal{T}_t$ и его ценой.

Сравнивать такие методы имеет смысл только при равном бюджете: одинаковое число токенов, одинаковые FLOPs или одинаковое время обучения — и по одной заранее выбранной validation-метрике. Иначе выигрыш легко получить просто за счёт того, что один из методов дольше считал.

Метод	Как преобразуется градиент	Цена и границы применения
AdamW	Покоординатный второй момент	Дешёвый базовый метод; связи между координатами не учитываются
Schedule-Free	Усреднение траектории параметров	Нужно переключать режимы train и eval
Muon	Ортогонализация матрицы momentum	В первую очередь 2D-веса скрытых слоёв
SOAP	Adam в собственном базисе накопленных статистик	Дополнительная память и разложения матриц

Таблица 3. Современные оптимизаторы: вид преобразования и его цена

13. Как выбирать оптимизатор

Однозначно лучшего оптимизатора нет: выбор проверяется по validation-метрике. Практические ориентиры:

- для Adam стартовый learning rate часто берут около $3 \cdot 10^{-4}$;
- для SGD пробуют 10^{-2} или 10^{-3} ;
- Adam часто быстрее достигает хорошего значения функции, но может сильнее переобучаться;
- SGD с Momentum остаётся важной точкой сравнения;
- распространённая современная отправная точка — AdamW + CosineAnnealing с Warmup.

Не стоит сразу перебирать все оптимизаторы. Lion, Signum, SOAP и другие варианты разумно проверять, если есть признаки плохой оптимизации или основные решения уже исчерпаны.



Что сравнивать

Сравнивают не только итоговую обучающую потерю, но и validation-качество, скорость выхода на нужный уровень, устойчивость обучения и вычислительную стоимость.

14. Главное

1. Полный градиент по большой выборке дорог, поэтому нейросети обычно обучают mini-batch-методами.
2. Стохастический градиент является несмещённой, но шумной оценкой полного градиента.
3. Постоянный learning rate оставляет метод в окрестности решения; уменьшение шага позволяет уточнить параметры.
4. Размер batch, learning rate и расписание определяют баланс между стоимостью, шумом и точностью.
5. Momentum сглаживает направление, а AdaGrad, RMSProp и Adam дополнительно масштабируют координаты.
6. В адаптивных методах L_2 -регуляризация и weight decay не эквивалентны; AdamW отделяет уменьшение параметров от адаптивного шага.
7. Плохая обусловленность объясняет зигзаги градиентного метода и мотивирует использование информации о кривизне.
8. Универсального оптимизатора нет: итоговый выбор делают по результатам на validation.



Ключевые формулы

Шаг mini-batch SGD

$$g_k = \frac{1}{|I_k|} \sum_{i \in I_k} \nabla L_i(w_k), \quad w_{k+1} = w_k - \alpha_k g_k.$$

Оценка при постоянном шаге

$$\mathbb{E}[F(\bar{w}_k)] - F(w^*) \leq \frac{R^2}{2h(k+1)} + \frac{hG^2}{2}.$$

Momentum

$$v_k = \beta v_{k-1} + (1 - \beta)g_k, \quad w_{k+1} = w_k - \alpha_k v_k.$$

Adam

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}, \quad w_{k+1} = w_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}.$$

AdamW

$$w_{k+1} = (1 - \alpha_k \lambda)w_k - \alpha_k \Delta_k.$$